

Chapter Four: Requirements Analysis

1. Introduction to Requirements Analysis

Requirements analysis is a fundamental phase of the software engineering process. It focuses on understanding **what the software system must do** and **the constraints under which it must operate**. The main goals of requirements analysis are:

- To **detect and resolve conflicts** between different requirements.
- To **discover the boundaries of the software system** and understand how it must interact with its **organizational and operational environment**.
- To **elaborate system-level requirements** and transform them into detailed **software requirements**.

This phase ensures that the developed software meets stakeholder needs and reduces the risk of costly errors later in development.

2. Requirements Classification

Requirements can be classified in several ways to improve understanding, management, and implementation.

2.1 Functional vs Non-Functional Requirements

- **Functional Requirements:** Describe what the system should do. They define system behavior, services, and functions.
 - Example: "The system shall allow users to search for a city."
- **Non-Functional Requirements:** Describe constraints or quality attributes of the system.
 - Examples: performance, security, usability, reliability, maintainability.

2.2 Source of Requirements

Requirements may originate from:

- High-level system requirements
- Stakeholders (users, customers, managers)
- Emergent properties of the system
- External constraints such as laws, standards, or policies

2.3 Product vs Process Requirements

- **Product Requirements:** Specify properties of the software product itself.

- **Process Requirements:** Specify constraints on how the software should be developed.
-

3. Requirement Priority

Not all requirements are equally important. Requirement prioritization helps determine which requirements are most critical.

Priority Levels

- **Mandatory:** Essential for system success
- **Highly Desirable:** Important but not strictly essential
- **Desirable:** Adds value but can be deferred
- **Optional:** Nice to have

Priority decisions must balance **importance**, **cost**, and **development effort**.

4. Scope of Requirements

The **scope** of a requirement refers to how broadly it affects the system.

- **Global Scope:** Affects the entire system architecture and many components.
- **Narrow Scope:** Affects a specific component and allows more design flexibility.

Understanding scope helps designers assess architectural impact and interdependencies.

5. Requirement Volatility

Requirement volatility refers to the likelihood that a requirement will change over time.

- Some requirements are stable.
- Others may change during development or the software lifecycle.

Causes of Volatility

- Changes in business needs
- Technological evolution
- Government laws or regulations

Estimating volatility helps in risk management and planning.

6. Conceptual Modeling

Conceptual modeling involves creating abstract models of real-world problems to aid understanding.

Purpose of Conceptual Models

- Understand the problem domain
- Represent real-world entities and their relationships
- Depict possible solutions

Conceptual models bridge the gap between real-world problems and software solutions.

7. Types of Conceptual Models

Several types of models may be developed during requirements analysis:

- Use Case Diagrams
- Data Flow Models
- State Models
- Goal-Based Models
- User Interaction Models
- Object Models
- Data Models

Many of these models are defined using the **Unified Modeling Language (UML)**.

8. Factors Influencing the Choice of Models

The selection of modeling techniques depends on several factors:

8.1 Nature of the Problem

- Real-time systems emphasize control flow and state models.
- Information systems emphasize data models.

8.2 Expertise of the Software Engineer

- Engineers tend to be more productive using modeling notations they are familiar with.

8.3 Customer Process Requirements

- Customers may require or prohibit specific notations or methods.
- This may conflict with the engineer's preferences.

8.4 Availability of Methods and Tools

- Poorly supported methods may not be widely adopted.
 - Tool and training availability strongly influence model choice.
-

9. Software Context Modeling

In most cases, it is useful to begin with a **software context model**.

Software Context

- Describes the relationship between the software system and its external environment.
- Identifies interfaces with external systems, users, and organizations.

Understanding the software context is critical to defining system boundaries and interactions.

10. Architectural Design and Requirements Allocation

10.1 Architectural Design

Architectural design defines:

- Hardware and software components
- Interfaces between components
- Overall system structure

Often, the software engineer also acts as the **software architect**.

10.2 Requirements Allocation

Requirements allocation assigns requirements to architectural components responsible for fulfilling them.

Benefits of allocation:

- Enables detailed analysis of component responsibilities
- Identifies component interactions
- Supports refinement of requirements

In large systems, allocation triggers further analysis for each subsystem.

Architectural design is closely related to conceptual modeling, but it is treated as a separate topic because mapping real-world entities to software components is not always straightforward.

11. Requirements Negotiation (Conflict Resolution)

Requirements negotiation addresses conflicts such as:

- Conflicting stakeholder demands
- Limited resources vs required functionality
- Conflicts between functional and non-functional requirements

Key Points

- Software engineers should avoid unilateral decisions.
 - Stakeholders must be consulted to reach a **consensus**.
 - Decisions should be **traceable to the customer** for contractual and accountability reasons.
-

12. Requirements Prioritization

Requirements prioritization helps to:

- Identify the most important requirements
- Resolve conflicts
- Plan staged or incremental deliveries

Analytic Hierarchy Process (AHP)

- Compares requirements pairwise
 - Determines which requirement has higher priority and by how much
 - Supports structured decision-making
-

13. Formal Analysis

Formal analysis is particularly useful in **high-integrity systems**.

Formal Requirements Expression

- Uses languages with formally defined semantics

Benefits

1. Eliminates ambiguity and misinterpretation
2. Enables formal reasoning and proof of system properties

Tool Support

Formal reasoning requires specialized tools:

- **Theorem Provers**
- **Model Checkers**

Proof cannot be fully automated, and the need for specialized expertise limits widespread adoption.

14. Summary

Requirements analysis is a critical phase that ensures software systems meet real-world needs. It involves classification, prioritization, modeling, negotiation, allocation, and formal analysis of requirements. A thorough understanding of these concepts leads to better software quality, reduced risk, and improved stakeholder satisfaction.

CHAPTER FIVE: REQUIREMENTS SPECIFICATION

1. Introduction to Software Requirements Specification (SRS)

Software Requirements Specification (SRS) is a formal document that clearly defines and describes both the **functional** and **non-functional requirements** of a software system. It serves as an official agreement and reference between stakeholders and the development team.

The SRS document ensures that all participants in the project have a **common understanding** of what the software system is expected to do and the constraints under which it must operate.

An SRS typically includes:

- Purpose of the system
 - System functionality
 - Performance requirements
 - Design constraints
 - User requirements
-

2. User Requirements

User requirements describe how the system should behave from the **end-user perspective**. They specify what features the system should have and how users will interact with the system.

User requirements are usually written in **natural language** and are intended to be easily understood by non-technical stakeholders.

3. Types of Requirements Documents

For complex systems, especially those involving **large non-software components**, three different types of requirements documents are produced:

1. **System Definition Document (SDD)**
2. **System Requirements Specification (SyRS)**
3. **Software Requirements Specification (SRS)**

For simple software systems, only the **Software Requirements Specification** is required.

4. System Definition Document (SDD)

The System Definition Document (SDD) is an essential component of the requirements specification process.

It provides a **high-level description** of the entire system from a domain perspective.

Contents of SDD:

- System architecture
- System objectives
- Design overview
- Implementation approach
- Target environment
- High-level system requirements

The SDD also defines the **data model**, which includes:

- Types of data stored
- Data organization
- Relationships between data elements

The SDD ensures that all stakeholders clearly understand the system structure and how it will be implemented.

5. System Requirements Specification (SyRS)

The System Requirements Specification (SyRS) is a **more detailed version** of the SDD.

It provides a complete description of:

- System functional requirements
- System non-functional requirements
- Design constraints and limitations

System requirements describe the system **as a whole**, including:

- Hardware requirements
- Software requirements
- Operational processes
- Documentation requirements

The SyRS typically includes:

- User interface requirements
 - Data storage and retrieval requirements
 - Security requirements
 - Performance requirements
-

6. Software Requirements Specification (SRS)

The Software Requirements Specification focuses only on the **software aspects** of the system.

It provides a realistic basis for:

- Cost estimation
- Risk assessment
- Schedule planning

Organizations also use the software SRS to develop **verification and validation (V&V) plans**.

7. Importance of Software Requirements Specification

Having a well-defined SRS is critical to the success of software development projects.

A good SRS:

- Ensures shared understanding among stakeholders
 - Reduces miscommunication
 - Improves development efficiency
 - Minimizes errors and rework
 - Produces higher-quality software
-

8. Conclusion (Chapter Five)

System requirements define the hardware and software environment needed to run the system, while software requirements define the features and functionality the software must provide.

CHAPTER SIX: REQUIREMENTS VALIDATION

1. Introduction to Requirements Validation

Requirements validation is the process of **checking that the documented requirements accurately define the system that the customer really wants.**

This process overlaps with requirements elicitation and requirements analysis.

Requirements validation is extremely important because **errors in requirements are very expensive to fix later.**

2. Cost of Requirements Errors

- Fixing a requirements error after delivery may cost **up to 100 times more** than fixing a coding error.
 - Changing requirements often requires:
 - Redesign
 - Reimplementation
 - Retesting
-

3. Purpose of Requirements Validation

- Ensure requirements reflect customer needs
 - Detect errors, omissions, and inconsistencies early
 - Reduce project risk and cost
-

4. Requirements Checking

During requirements validation, several types of checks are performed:

4.1 Validity Checks

Ensure requirements represent the real needs of users and stakeholders.

4.2 Consistency Checks

Ensure there are no conflicting or contradictory requirements.

4.3 Completeness Checks

Ensure all required functions and constraints are specified.

4.4 Realism Checks

Ensure requirements are feasible within budget, schedule, and available technology.

4.5 Verifiability Checks

Ensure each requirement can be tested and verified.

5. Summary of Validation Criteria

- Validity
 - Consistency
 - Completeness
 - Realism
 - Verifiability
-

6. Requirements Validation Techniques

6.1 Requirements Reviews

A systematic manual examination of requirements by reviewers to detect errors.

6.2 Prototyping

Development of an executable model to allow users to evaluate system behavior and provide feedback.

6.3 Test-Case Generation

Developing test cases from requirements to check their testability.

7. Requirements Changes and Problems

It is rare to identify all requirements problems during validation.

Additional changes often occur due to:

- Misunderstandings
 - Omissions
 - Evolving customer needs
-

8. Requirements Reviews – Golden Rules

- Conduct reviews regularly
 - Involve both client and contractor
 - Reviews may be formal or informal
 - Good communication is essential
-

9. Review Checks

- Verifiability
 - Comprehensibility
 - Traceability
 - Adaptability
-

10. Conclusion (Chapter Six)

Requirements validation ensures that requirements are complete, correct, and consistent. It must be iterative and involve all stakeholders.

CHAPTER SEVEN: PRACTICAL CONSIDERATIONS

1. Iterative Nature of the Requirements Process

The requirements process must be iterative to accommodate feedback and change.

Iteration allows:

- Continuous validation
 - Refinement of requirements
 - Improved product quality
-

2. Importance of Iteration

Without iteration:

- Requirements may be misunderstood
 - Errors may be discovered too late
 - Projects may fail
-

3. Benefits of Iterative Approach

- Early stakeholder feedback
 - Detection of issues early
 - Continuous improvement
 - Better collaboration and communication
-

4. Change Management

Change management deals with managing changes to system requirements.

It involves:

- Procedures
 - Processes
 - Standards
-

5. Change Management Policies

Change management policies may include:

- Change request procedures
 - Impact analysis methods
 - Cost estimation
 - Traceability information
 - Tool support
-

6. Change Request Status

A change request may have statuses such as:

- Proposed
 - Accepted
 - Rejected
 - Included
-

7. Change Management Process

1. Identify requirements problem
2. Analyze problem
3. Propose changes
4. Analyze impact and cost

5. Implement change
 6. Update documentation
-

8. Change Request Form

A change request form includes:

- Date
 - Customer
 - Requester
 - Product and version
 - Description and rationale
 - Impact analysis
 - Signatures
 - Status and comments
-

9. Requirements Attributes

Requirements attributes describe the characteristics of requirements.

These include:

- Completeness
- Correctness
- Consistency
- Feasibility
- Priority
- Traceability
- Testability
- Verifiability

Additional attributes may include:

- Requirement source
 - Rationale
 - Change history
-

10. Requirements Tracing

Requirements tracing documents relationships between requirements and other artifacts.

11. Types of Requirements Tracing

- Forward tracing
 - Backward tracing
 - Bidirectional tracing
-

12. Traceability Matrix

A traceability matrix defines links between:

- Requirements
 - Design elements
 - Use cases
 - Test cases
-

13. Measuring Requirements

Measuring requirements helps estimate software size.

14. Functional Size Measurement (FSM)

FSM measures software size based on functional requirements.

Function points are calculated using:

- Inputs
- Outputs
- Inquiries
- Internal files
- External interfaces

FSM is used for:

- Estimation
- Planning
- Project control

15. Conclusion (Chapter Seven)

Practical considerations ensure effective management of requirements throughout the software lifecycle.

END OF CHAPTERS FIVE, SIX, AND SEVEN